Code Seed

LECTURE 03 · FOUNDATIONS

Databases & Data Modeling

Storing, connecting, and searching what we know.

03

Understand the concept. **Then build it.**

THE FOUNDATIONS

Three ideas before we build

DONE

LECTURE 01

How the Web Works

Client · Server · HTTP

DONE

LECTURE 02

APIs & REST

Endpoints · CRUD · JSON

YOU ARE HERE

LECTURE 03

Databases

Tables · Relations · SQL · Search

Then → the coding series

THINK ABOUT IT

The server restarts. Where did all the notes go?

In Lecture 01 our server kept notes in a JavaScript variable. Variables live in memory — and memory is wiped on every restart.

01

Why Databases

Memory forgets. Disks remember.

PERSISTENCE

Data that survives a restart

Memory (RAM)

Fast, but wiped on every restart or crash

Disk

Slower, but permanent

A database

Software that manages data on disk — safely and fast

Stored in	Before restart	After restart
A variable	3 notes	0 notes
A database	3 notes	3 notes

DESIGN QUESTION

Why not just save a JSON file?

✕ NOTES.JSON

A file

- Two saves at once → one is lost
- Read the whole file to find one note
- No rules — bad data slips in
- Slower and slower as it grows

✓ DATABASE

A database

- Many users at the same time
- Finds one row among millions
- Enforces rules on every write
- Search, backups, recovery built in

TWO FAMILIES

Relational vs. document databases

✓ SQL

Relational

Tables with fixed columns
Like spreadsheets with strict rules

Relationships built in
Notes ↔ tags ↔ users

PostgreSQL · MySQL · SQLite

{ } NOSQL

Document

Flexible JSON documents
Each record can have its own shape

Relationships are harder
Often duplicated or linked by hand

MongoDB · Firestore · DynamoDB

Neither is “better”. Our data is highly connected — so we choose relational.

OUR CHOICE

In our Knowledge Base

Why PostgreSQL

Relationships

Notes, tags and users — connected and enforced

Full-text search

Search notes and PDF text, built in

JSON when needed

JSONB columns for flexible extras

Free, everywhere

Open source, supported by every host

02

The Relational Model

Tables, rows, columns, and keys.

TABLES

In our Knowledge Base

A table is a spreadsheet with rules

notes

id	type	title	language	created_at
40	concept	SQL Joins	NULL	2026-10-28 14:02
41	snippet	Debounce in JS	javascript	2026-10-30 09:15
42	concept	Idempotency	NULL	2026-10-31 09:30

Each row is one note. Each column is one property, with a fixed type.

PRIMARY KEY

Every row needs an identity

Unique

No two rows share it

Never changes

Titles change. Ids don’t.

Connects the layers

`/api/notes/42` → `WHERE id = 42`

id	title
40	SQL Joins
41	Debounce in JS
42	Idempotency

03

Modeling Our Knowledge Base

From features to tables.

In our Knowledge Base

FIND THE NOUNS

A user writes notes, labels them with tags, and uploads files.

Step 1: find the nouns. Each becomes a table — the same nouns as our API resources.

RELATIONSHIPS

Three kinds of relationships

One-to-one

A user has one settings record

`1 : 1`

One-to-many

A user has many notes. Each note has one owner.

`1 : N`

Many-to-many

A note has many tags. A tag is on many notes.

`N : M`

ONE-TO-MANY

In our Knowledge Base

One-to-many: the foreign key

Foreign key

A column pointing to another table’s primary key

The “many” side holds it

Each note stores its `user_id`

The database enforces it

No note can point to a user who doesn’t exist

users

id	email
1	ada@example.com
2	linus@example.com

↑ `user_id references users.id`

notes

id	user_id	title
41	1	Debounce in JS
42	1	Idempotency
43	2	Git rebase

MANY-TO-MANY

In our Knowledge Base

Many-to-many: a junction table

notes

id	title
41	Debounce in JS
42	Idempotency

note_tags

note_id	tag_id
42	1
42	2
41	3
41	1

tags

id	name
1	http
2	api
3	javascript

Each row in `note_tags` is one connection: “note 42 has tag 1”.

THINK ABOUT IT

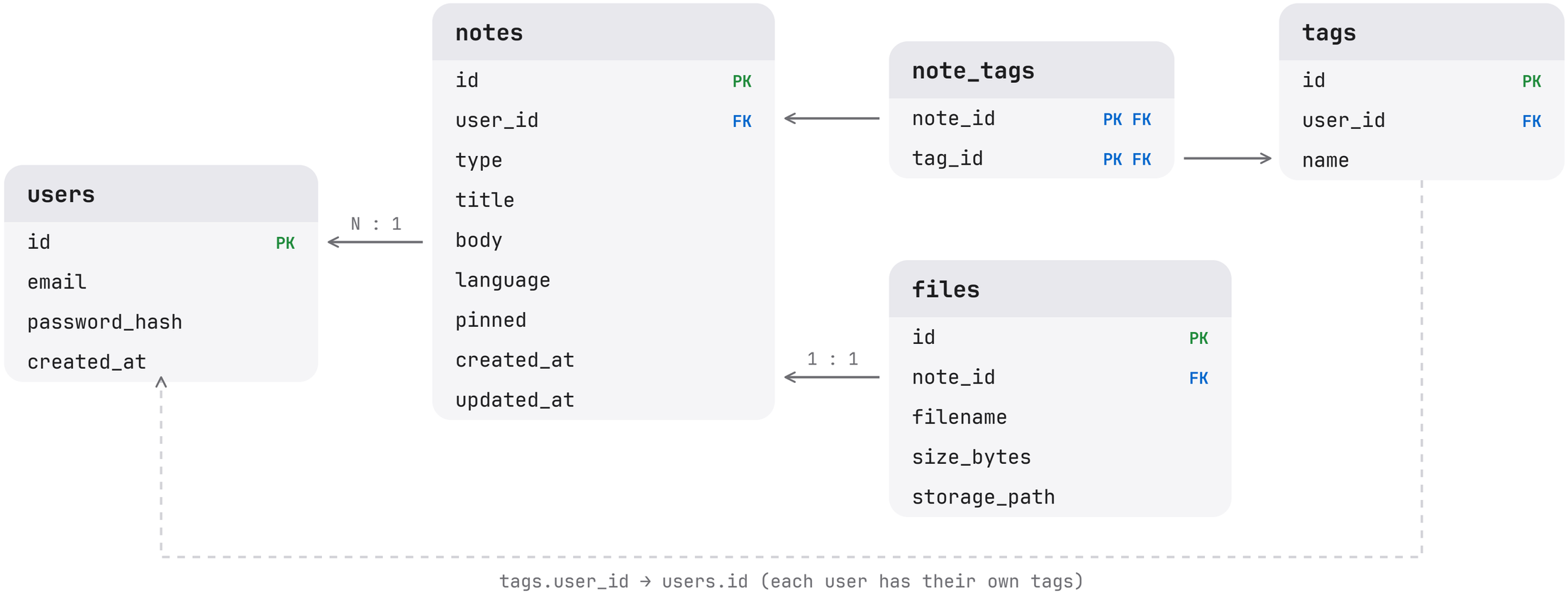
Notes, snippets, concepts, PDFs: four tables — or one?

What do they share? What is different? What happens when we tag or search across all of them?

THE SCHEMA

In our Knowledge Base

Our Knowledge Base, in five tables



In our Knowledge Base

Creating the notes table

schema.sql SQL

```sql
CREATE TABLE notes (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type        TEXT NOT NULL CHECK (type IN ('note', 'snippet', 'concept', 'pdf')),
  title       TEXT NOT NULL,
  body        TEXT NOT NULL DEFAULT '',   -- for PDFs: the extracted text
  language    TEXT,                       -- only used by snippets
  pinned      BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

SQL · SCHEMA

Tags and the junction table

schema.sql SQL

```sql
CREATE TABLE tags (
  id      BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name    TEXT NOT NULL,
  UNIQUE (user_id, name)              -- no duplicate tags per user
);

CREATE TABLE note_tags (
  note_id BIGINT REFERENCES notes(id) ON DELETE CASCADE,
  tag_id  BIGINT REFERENCES tags(id)  ON DELETE CASCADE,
  PRIMARY KEY (note_id, tag_id)       -- each pair only once
);
```

CONSTRAINTS

In our Knowledge Base

Rules the database enforces

CONSTRAINT	MEANS	IN OUR KNOWLEDGE BASE
`NOT NULL`	A value must exist	Every note has a title
`UNIQUE`	No duplicates	One account per email
`PRIMARY KEY`	Unique, non-null identity	`notes.id`
`REFERENCES`	Must point to a real row	`notes.user_id` → `users.id`
`CHECK`	A custom rule	`type` is note, snippet, concept or pdf
`ON DELETE CASCADE`	Delete the children too	Delete a user → their notes go too

04

SQL: Talking to the Database

CRUD, one last time.

CREATE

In our Knowledge Base

INSERT — save a note

query.sql SQL

```sql
INSERT INTO notes (user_id, type, title, body)
VALUES (1, 'concept', 'Idempotency', 'Same request, same result…')
RETURNING id, created_at;

-- id | created_at
-- 42 | 2026-10-31 09:30:00+00
```

`INSERT INTO`

Create a new row

`RETURNING`

Hand back what the database generated

Maps to

`POST /api/notes` → `201 Created`

READ

In our Knowledge Base

SELECT — find notes

query.sql SQL

```sql
SELECT id, type, title, updated_at
FROM notes
WHERE user_id = 1
  AND type = 'snippet'
ORDER BY updated_at DESC
LIMIT 20 OFFSET 20;          -- page 2
```

`WHERE`
Filter rows — like `?type=snippet`

`ORDER BY`
Sort — like `?sort=-updatedAt`

`LIMIT` / `OFFSET`
Pagination — from Lecture 02

UPDATE and DELETE

query.sql SQL

```sql
UPDATE notes
SET pinned = true, updated_at = now()
WHERE id = 42 AND user_id = 1;

DELETE FROM notes
WHERE id = 42 AND user_id = 1;
```

`PATCH /api/notes/42`

→ `UPDATE … WHERE id = 42`

`DELETE /api/notes/42`

→ `DELETE … WHERE id = 42`

Never forget `WHERE`

Without it, you change every row

JOINS

JOIN — combine tables

query.sql SQL

```sql
SELECT n.title, t.name AS tag
FROM notes n
JOIN note_tags nt ON nt.note_id = n.id
JOIN tags t       ON t.id = nt.tag_id
WHERE n.id = 42;
```

result

title	tag
Idempotency	http
Idempotency	api

THINK ABOUT IT

How would you find every note tagged sql?

Which tables do you need? Which columns connect them? Write the query before moving on.

05

Speed, Search & Files

Making it fast — and findable.

Indexes: the back of the book

Without an index

Scan every row — O(n)

With an index

Jump straight there — O(log n)

The trade-off

Faster reads, slightly slower writes and more disk

NO INDEX

1,000,000

rows checked

WITH INDEX

~20

steps · $\log_2(10^6) \approx 20$

index.sql — SQL

```sql
CREATE INDEX notes_user_idx ON notes (user_id);
```

FULL-TEXT SEARCH

Search words, not characters

`LIKE '%index%'`

Scans every row · no ranking · misses “indexes”

`tsvector`

Text reduced to searchable word stems

`tsquery`

The search terms, reduced the same way

`@@`

“Does this document match this query?”

psql SQL

```sql
SELECT to_tsvector('english',
  'Searching notes and indexes');

-- 'index':4 'note':2 'search':1
```

SEARCH

In our Knowledge Base

Search in our Knowledge Base

search.sql — SQL

```sql
ALTER TABLE notes ADD COLUMN search tsvector
  GENERATED ALWAYS AS
    (to_tsvector('english', title || ' ' || body)) STORED;

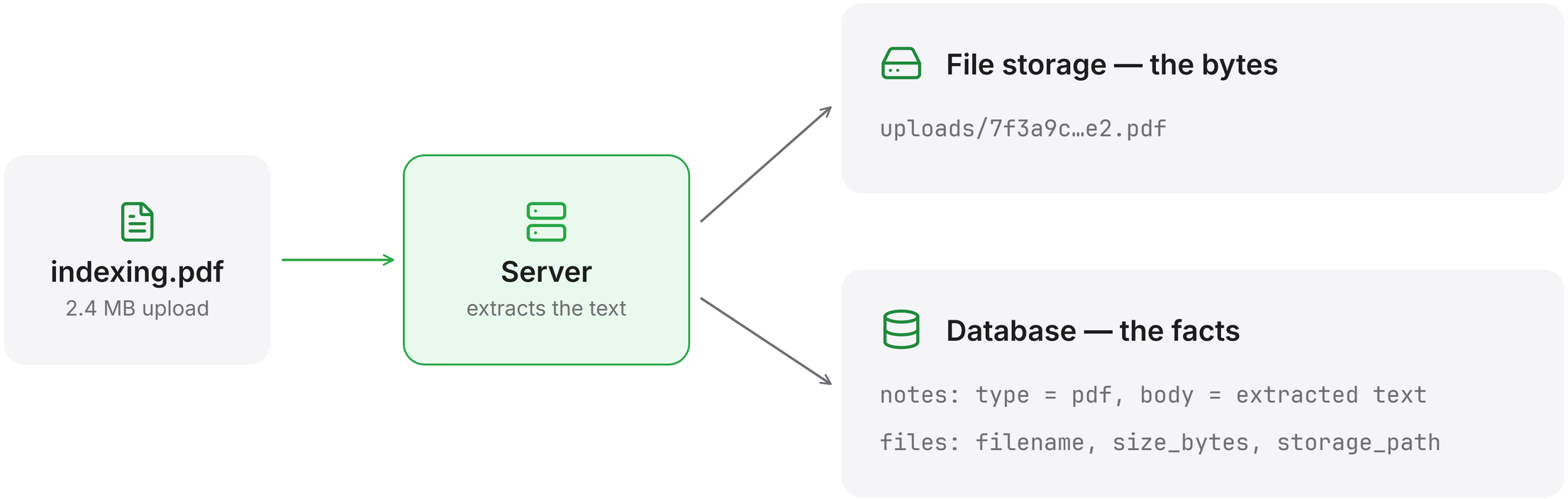
CREATE INDEX notes_search_idx ON notes USING GIN (search);

SELECT id, type, title, ts_rank(search, q) AS rank
FROM notes, websearch_to_tsquery('english', 'react hooks') q
WHERE user_id = 1 AND search @@ q
ORDER BY rank DESC LIMIT 20;
```

FILES

In our Knowledge Base

Storing PDFs: bytes vs. facts



Rule of thumb: keep big files outside the database. Keep their path and text inside it.

SECURITY

Never build SQL from user input

✕ INJECTABLE

String building

JAVASCRIPT

```
const sql = `SELECT * FROM notes
  WHERE title = '${req.query.q}'`;
```

If `q` is `x' OR '1'='1` the query returns every user's notes.

✓ SAFE

Parameterized query

JAVASCRIPT

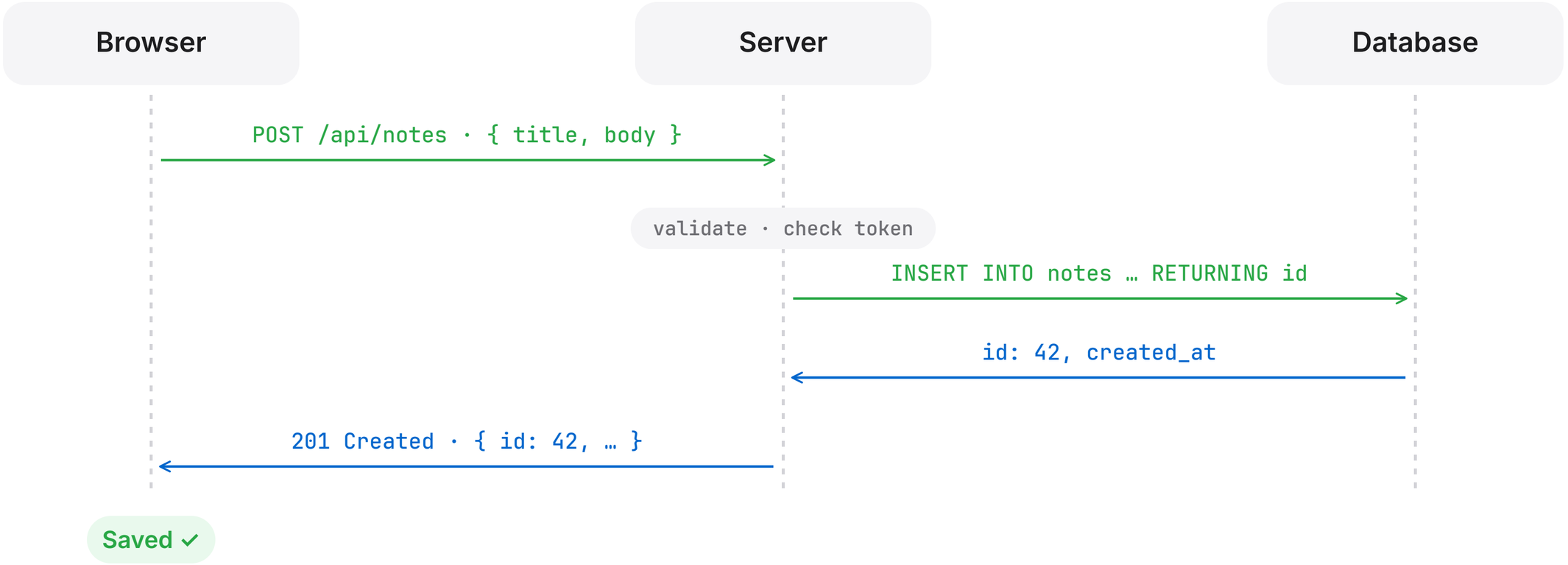
```
await pool.query(
  'SELECT * FROM notes WHERE title = $1',
  [req.query.q]
);
```

Values are sent separately from the SQL. Input can never change the query.

FULL STACK

In our Knowledge Base

One click, every layer



RECAP

Three lectures, one mental model

01 **Client ↔ Server over HTTP**
Requests, responses, status codes, JSON

02 **APIs are contracts**
REST resources + CRUD methods

03 **Databases remember**
Tables, rows, columns, primary keys

04 **Relationships are data**
Foreign keys and junction tables

05 **SQL mirrors CRUD**
INSERT · SELECT · UPDATE · DELETE

06 **Fast, findable, safe**
Indexes · full-text search · parameterized queries

COMING NEXT

In our Knowledge Base

The coding series

01 **Project setup**
Node.js, Express, Git, folder structure

02 **First server**
Routes, JSON, testing with curl

03 **The database**
PostgreSQL, schema, migrations

04 **Notes API**
Full CRUD, backed by SQL

05 **The frontend**
List, create, and edit notes

06 **Tags**
Many-to-many in practice

07 **Search**
Full-text search, end to end

08 **PDF uploads**
Storage and text extraction

09 **Authentication**
Sign up, log in, tokens

10 **Deploy**
HTTPS, a domain, production database



THE FOUNDATIONS ARE PLANTED

Now we build it.

You understand the concepts. In Episode 01 we set up the project and write our first line of code.

Subscribe · Episode 01: Project Setup